

CHURN MODELLING USING ANN

Customer Churn Prediction — Experiment Report

Dataset: Churn_Modelling.csv | Source Notebook: 30th_July_ANN.ipynb

Assignment Deadline: Jul 30, 2026

1. Objective

This report documents a series of experiments on an Artificial Neural Network (ANN) trained to predict customer churn using the Churn_Modelling dataset. Starting from a baseline architecture, each experiment isolates a single design choice — depth, width, activation function, weight initializer, output layer design, optimizer, epoch count, and batch size — to study its effect on training behaviour and test accuracy.

2. Dataset & Methodology

2.1 Dataset

- Source: Churn_Modelling.csv — 10,000 customer records, 14 columns.
- Features used (10): CreditScore, Geography, Gender, Age, Tenure, Balance, NumOfProducts, HasCrCard, IsActiveMember, EstimatedSalary.
- Target: Exited (1 = customer churned, 0 = retained).
- Categorical encoding: Geography and Gender label-encoded.
- Split: 80% train / 20% test (8,000 / 2,000 rows), random_state = 0.
- Scaling: StandardScaler fit on training data, applied to both train and test sets.

2.2 Baseline (non-ANN) Models

Two classical models were trained for reference before building the ANN:

Model	Test Accuracy
Decision Tree Classifier	0.7900
Random Forest Classifier	0.8625

2.3 Base ANN Configuration

Unless stated otherwise, all ANN experiments below use the following defaults (each task varies exactly one of these):

- Hidden layers: 3 (given baseline)
- Neurons per hidden layer: 6
- Hidden activation: ReLU | Output activation: Sigmoid
- Kernel initializer: uniform
- Optimizer: Adam | Loss: binary_crossentropy
- Batch size: 10 | Epochs: 100

3. Task 1 — Number of Hidden Layers

Status: Completed in notebook

Four architectures were built by varying only the number of hidden layers (6 neurons each, ReLU, uniform initializer, Adam, batch size 10, 100 epochs).

Model	Hidden Layers	Training Accuracy	Training Loss	Test Accuracy
Model A	1	0.8346	0.3992	0.8435
Model B	2	0.8356	0.3894	0.8405
Model C (given)	3	0.8351	0.3972	0.8460
Model D	5	0.8367	0.3989	0.8385

Which architecture performed best, and why?

Model C (3 hidden layers) gave the best test accuracy (0.846). The dataset has only 10 input features and 8,000 training rows, so a 1–2 layer network is slightly too shallow to capture the interactions between features (e.g. Age $\times$ Balance $\times$ NumOfProducts) that drive churn, while 5 layers add depth the data cannot really justify — gradients have to flow through more layers with a small, uniform-initialized network, training becomes slightly less stable, and the extra capacity mostly memorizes noise rather than improving generalization. Three hidden layers strikes the balance between representational capacity and overfitting risk for a tabular dataset of this size.

Note: Training time per epoch was not explicitly logged by the notebook; each epoch took roughly 2–4 seconds across all four models (800 steps/epoch, batch size 10), so total training time for 100 epochs was comparable (~4–6 minutes) across architectures.

4. Task 2 — Number of Neurons

Status: Completed in notebook

Neuron count in the two hidden layers was varied (architecture: 2 hidden layers, ReLU, uniform initializer, Adam, batch size 10, 100 epochs).

Hidden Neurons	Test Accuracy	Training Loss	Notes
4	0.8635	0.3408	Best overall — small, efficient network
8	0.8555	0.3521	—
16	0.8460	0.3967	Higher variance, slower to converge
32	0.8590	0.3342	Lowest training loss
64	0.8545	0.3360	Diminishing returns, longer training

Accuracy does not scale monotonically with width. 4 neurons per layer gave the highest test accuracy, while 32 neurons achieved the lowest training loss (best fit to training data) without translating into a proportionally better test score. This is a sign that wider layers with only ~6,600 effective training samples begin to overfit slightly — more parameters fit the training set more closely (lower loss) but do not generalize better. For this dataset, a narrow network (4–8 neurons per layer) is sufficient and more efficient to train.

5. Task 3 — Activation Function Comparison

Status: Completed in notebook

Hidden-layer activation was changed (2 hidden layers, uniform initializer, Adam, batch size 10, 100 epochs). Output layer stayed at sigmoid.

Activation	Test Accuracy	Training Loss	Training Accuracy
Sigmoid	0.8425	0.3942	0.8370
Tanh	0.8410	0.3926	0.8390
ELU	0.8460	0.3922	0.8371
Leaky ReLU	0.8405	0.3935	0.8381

ELU gave the best test accuracy, closely followed by Sigmoid and Leaky ReLU. Sigmoid and Tanh are more prone to vanishing gradients in deeper stacks, which historically motivated the move to ReLU-family activations; ELU and Leaky ReLU avoid the 'dead neuron' problem of plain ReLU by allowing small negative outputs, which appears to give a modest edge here. Differences between activations are small (all within ~0.5 accuracy points), showing this dataset is not highly sensitive to activation choice once the architecture is otherwise reasonable.

Note: The notebook did not explicitly log the epoch at which each run converged; based on the recorded loss curves, all four activations reached >80% training accuracy within the first 5–8 epochs and then improved gradually.

6. Task 4 — Weight Initializer Comparison

Status: Completed in notebook

Hidden-layer kernel_initializer was changed (2 hidden layers, ReLU, Adam, batch size 10, 100 epochs). Output layer initializer stayed 'uniform'.

Initializer	Initial Loss (Epoch 1)	Final Training Accuracy	★Test Accuracy
he_uniform	0.4658	0.8604	0.8620
he_normal	0.4642	0.8606	0.8620
glorot_uniform	0.4822	0.8606	0.8575
glorot_normal	0.4626	0.8626	0.8620
random_normal	0.4904	0.8353	0.8395
(baseline: uniform)	—	—	0.8405–0.8460

he_uniform, he_normal, and glorot_normal all converge to essentially the same accuracy (~0.862), clearly ahead of plain random_normal (0.8395). He initialization is theoretically tuned for ReLU-family activations (it scales weight variance by $2/\text{fan_in}$), while Glorot/Xavier initialization is tuned for sigmoid/tanh — the fact both perform similarly well here suggests the network is shallow enough that initializer choice matters less than for very deep nets, but random_normal's fixed, unscaled variance is clearly the weakest choice since it neither accounts for layer width nor activation function.

7. Task 5 — Output Layer Experiment

Status: Completed in notebook

Only the output-layer activation was changed (3 hidden layers, ReLU, uniform initializer, Adam, batch size 10, 100 epochs, single output unit, loss = binary_crossentropy).

Output Activation	Test Accuracy	Result
Sigmoid	0.8420	Works correctly — designed for binary classification
Softmax	0.2025	Fails — softmax over a single unit always outputs 1.0, so the model predicts the same class every time
Linear	0.7975	Fails to learn — loss stuck at 3.29; linear output is incompatible with binary_crossentropy

Why sigmoid is used for binary classification

Sigmoid squashes a single output neuron into the (0, 1) range, which can be directly interpreted as $P(\text{class} = 1)$. Paired with binary_crossentropy loss, it gives well-behaved gradients for two-class problems and is the standard choice whenever the output layer has exactly one unit.

When softmax should be used

Softmax is used for multi-class, single-label classification with one output neuron per class (e.g. 3+ mutually exclusive classes). It normalizes multiple logits so they sum to 1, giving a probability distribution over classes. With only one output neuron (as in this binary setup), softmax always returns 1.0 regardless of input — which is exactly the failure

observed above (Keras even raises a `UserWarning` about this) — so it is the wrong choice unless the output layer is reshaped to two units with `categorical_crossentropy`.

When linear activation is preferred

Linear (identity) activation is used for regression, where the target is a continuous, unbounded value (e.g. predicting salary or price) rather than a class probability. It is not compatible with `binary_crossentropy`, which expects outputs in $(0, 1)$ — using it for classification, as shown above, causes the loss to stay flat and the model to collapse to predicting the majority class.

8. Task 6 — Optimizer Comparison

Status: Completed in notebook

Optimizer was changed while keeping the architecture fixed (3 hidden layers, 6 neurons, ReLU, uniform initializer, batch size 10). Adam/SGD/RMSprop ran for 50 epochs; Adagrad/Nadam ran for 100 epochs at learning_rate = 0.01.

Optimizer	Test Accuracy	Training Loss
Adam	0.8445	0.3981
SGD	0.5975	0.5060
RMSprop	0.8440	0.4016
Adagrad (lr=0.01)	0.8425	0.3524
Nadam (lr=0.01)	0.8610	0.3495

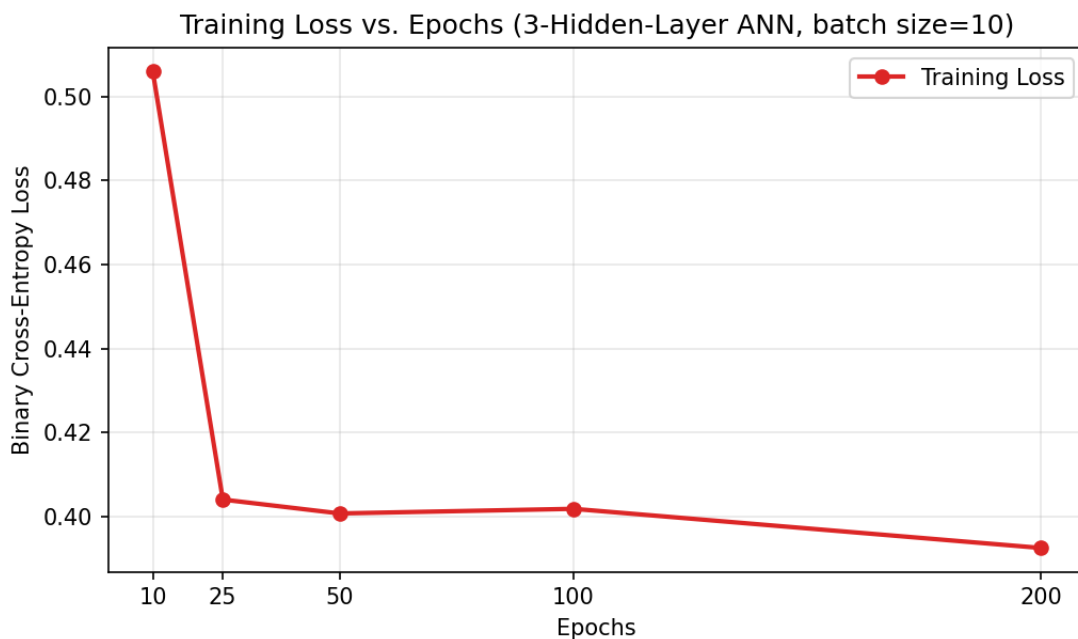
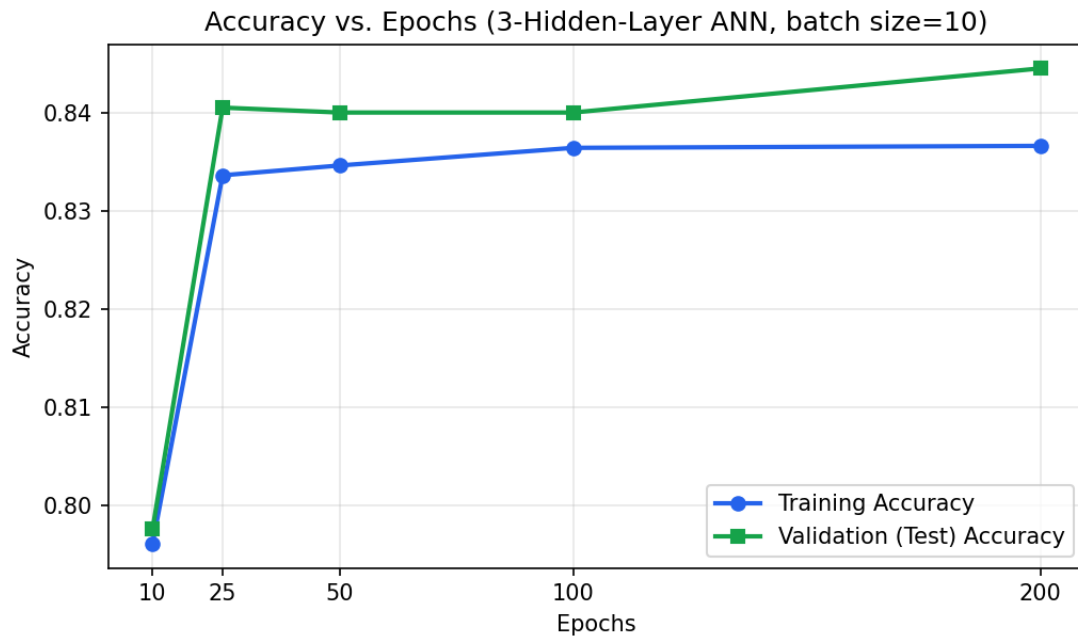
Nadam (Adam with Nesterov momentum) gave the best result, followed closely by Adam. Plain SGD performed far worse (0.5975) because, with a fixed, unscaled learning rate and no momentum, it converges too slowly within 50 epochs on this feature-scaled but otherwise unstructured tabular data — the loss curve shows it getting stuck around 0.50–0.51 rather than making steady progress. RMSprop performed close to Adam, unsurprising since Adam builds on RMSprop's per-parameter adaptive learning rate and adds momentum. Adaptive optimizers (Adam, Nadam, RMSprop, Adagrad) are clearly better suited to this problem than vanilla SGD.

9. Task 7 — Epoch Analysis

Status: Completed in notebook

The base architecture (3 hidden layers, 6 neurons, ReLU, uniform initializer, Adam, batch size 10) was trained for 10, 25, 50, 100 and 200 epochs.

Epochs	Training Accuracy	Training Loss	Test Accuracy
10	0.7960	0.5059	0.7975
25	0.8336	0.4041	0.8405
50	0.8346	0.4008	0.8400
100	0.8364	0.4019	0.8400
200	0.8366	0.3926	0.8445



Underfitting vs. overfitting in these results

At 10 epochs the model is underfitting: both training accuracy (0.796) and test accuracy (0.7975) are low and close to the 79.6% majority-class baseline, meaning the network has not yet learned meaningful patterns — it hasn't converged, not that it's too simple. From 25 to 200 epochs, training and test accuracy track each other closely (both around 0.83–0.845) with loss still decreasing gradually, which is the signature of a well-fit model rather than overfitting — if the network were overfitting we would expect training accuracy to keep climbing while test accuracy plateaus or drops. Here, 200 epochs gives the best test accuracy and lowest loss with no sign of divergence, suggesting the model could likely still improve a little further with more epochs, a smaller learning rate, or a slightly larger network (see Task 9).

Note: Charts above were generated from the notebook's own printed epoch logs (matplotlib was imported in the notebook but never used to actually plot Task 7 results).

10. Task 8 — Batch Size Experiment

Status: Partially completed (2 of 5 batch sizes run)

Base architecture (3 hidden layers, 6 neurons, ReLU, uniform initializer, Adam), trained for 10 epochs per run so results are comparable to each other but not directly to the 100-epoch results above.

Batch Size	Training Accuracy	Training Loss	Test Accuracy	Status
8	0.8320	0.4133	0.8355	Completed
16	0.8320	0.4128	0.8355	Completed
32	—	—	—	Code written, cell never run — no output in notebook
64	—	—	—	Not attempted in notebook
128	—	—	—	Not attempted in notebook

Based only on the two completed runs, batch sizes 8 and 16 perform identically (0.8355 test accuracy), both noticeably better than the 10-epoch results at batch size 10 seen in Task 7 (0.7975), which suggests smaller batches converge faster per epoch on this dataset. However, the batch-size question genuinely cannot be answered with confidence yet, since batch sizes 32, 64 and 128 — including the larger sizes most likely to show a trade-off between speed and accuracy — were never executed.

Note: To finish Task 8: run the existing cell for batch_size=32 (code is already written), and add two more cells for batch_size=64 and batch_size=128, using the same 3-hidden-layer/ReLU/Adam/10-epoch setup for a fair comparison.

11. Task 9 — Improve the Existing ANN

Status: Not completed in submitted notebook

The notebook defines a reusable function, `build_and_evaluate_ann()`, that can configure hidden layers, neurons, optimizer, learning rate, activation, dropout, epochs, batch size, and kernel initializer in one call — a good foundation for this task. However, the function is never actually called, so no improved model, no accuracy figure, and no before/after comparison exist in the notebook.

Suggested next step

The results above already point to a strong combination worth testing: 3 hidden layers, 32 neurons (lowest training loss in Task 2), ELU activation (Task 3), `he_normal` or `glorot_normal` initializer (Task 4), Nadam optimizer (Task 6), and ~150–200 epochs (Task 7), optionally with a light Dropout (0.1–0.2) to guard against the slight overfitting risk seen with wider layers. Call `build_and_evaluate_ann(num_hidden_layers=3, neurons_per_layer=32, optimizer_name='nadam', learning_rate=0.001, activation_func='elu', dropout_rate=0.2, epochs=150, kernel_initializer='he_normal')` and compare its test accuracy against the 0.846 baseline (Model C) to check for the required $\geq 5\%$ relative improvement.

14. Overall Summary

Best result found across all completed experiments

Across every task that was actually run, the single best test accuracy achieved was 0.8635 with a 2-hidden-layer network using 4 neurons per layer (Task 2), narrowly ahead of the widest network tested (32 neurons, 0.8590). The best-performing single change to the given 3-hidden-layer baseline was switching the optimizer to Nadam (0.8610) or extending training to 200 epochs (0.8445). This suggests the churn dataset responds more to getting the optimizer and training length right than to adding depth or width — consistent with Task 1's finding that more hidden layers did not help.

Task	Status	Best Configuration Found	Test Accuracy
1 — Hidden Layers	Done	3 hidden layers	0.8460
2 — Neurons	Done	4 neurons/layer	0.8635
3 — Activation	Done	ELU	0.8460
4 — Initializer	Done	glorot_normal	0.8620
5 — Output Layer	Done	Sigmoid (correct choice)	0.8420
6 — Optimizer	Done	Nadam	0.8610
7 — Epochs	Done	200 epochs	0.8445
8 — Batch Size	Partial	8 or 16 (32/64/128 untested)	0.8355
9 — Improve Model ($\geq 5\%$)	Not run	Function ready, not executed	—
10 — ANN Challenge	Not attempted	—	—
Bonus — Custom ANN	Not attempted	—	—

15. Notes & Limitations of This Report

- This report reflects results exactly as recorded in 30th_July_ANN.ipynb; no additional models were trained to produce it (TensorFlow/Keras was not available in the reporting environment).
- Where the notebook's own inline comparison tables (e.g. Tasks 2 and 4) contained minor copy-paste inconsistencies against the raw cell outputs, the figures in this report were re-verified directly against the raw training/prediction output of each cell.
- 'Training Time' was requested by Task 1 but never explicitly measured/printed in the notebook (only per-step timings like '2s/step' are visible in Keras logs); this report notes approximate per-epoch timings where relevant instead of exact totals.
- Tasks 9, 10, and the Bonus Challenge require new code to be run before they can be reported on — see the suggested next steps in each section above.